

VL-12: P und NP

(Berechenbarkeit und Komplexität, WS 2024)

Peter Rossmanith

WS 2024, RWTH

Zusammenfassung: Berechenbarkeit

Zusammenfassung: Turing-Mächtigkeit

Wir haben die folgenden **Turing-mächtigen** Rechenmodelle und Programmiersprachen kennengelernt:

- Turingmaschine (TM)
- k -Band-TM
- Registermaschine (RAM)
- WHILE-Programme (und somit C, Java, Pascal, Postscript, etc.)
- μ -rekursive Funktionen (und somit LISP, Haskell, OCaml, etc)

Die folgenden Rechenmodelle sind hingegen **nicht Turing-mächtig**:

- LOOP-Programme
- Primitiv rekursive Funktionen

Zusammenfassung: Church-Turing These

Church-Turing These

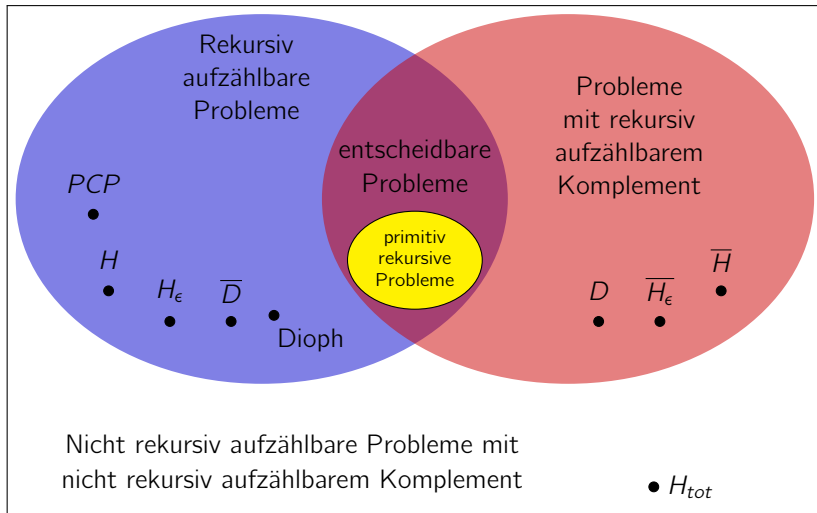
Die Klasse der TM-berechenbaren Funktionen stimmt mit der Klasse der “intuitiv berechenbaren” Funktionen überein.

In anderen Worten:

Ein Problem kann genau dann “algorithmisch gelöst werden”, wenn es eine TM für dieses Problem gibt.

An Stelle von “TM” können wir auch jedes andere Turing-mächtige Rechenmodell in die obigen Sätze einfügen.

Zusammenfassung: Berechenbarkeitslandschaft



Zusammenfassung: Unentscheidbarkeit (1)

Methoden zum Nachweis von Nicht-Berechenbarkeit:

- Diagonalisierung
- Unterprogrammtechnik
- Reduktionen (spezielle Variante der Unterprogrammtechnik)
- **Satz von Rice:** Aussagen über Eigenschaften von Funktionen, die durch eine gegebene TM berechnet werden, sind nicht entscheidbar

Schlussfolgerung aus Rice: Die automatische Verifikation von Programmen in T-mächtigen Programmiersprachen ist unmöglich

Zusammenfassung: Unentscheidbarkeit (2)

Wichtige nicht berechenbare Probleme:

- Halteproblem, in verschiedenen Varianten
- Postsches Korrespondenzproblem
- Verschiedene Entscheidungsprobleme für CFGs
- Erkennung von Funktionen mit elementaren Stammfunktionen
- Hilberts zehntes Problem

Wiederholung

Modelle für die Rechenzeit einer RAM

- **Uniformes Kostenmaß:**

Jeder Schritt der RAM zählt als eine Zeiteinheit.

- **Logarithmisches Kostenmaß:**

Die Laufzeitkosten eines Schrittes der RAM sind proportional zur binären Länge der Zahlen in den angesprochenen Registern.

Wdh.: Simulationen zwischen RAM und TM

Satz

Für jede im logarithmischen Kostenmass $t(n)$ -zeitbeschränkte RAM R gibt es ein Polynom q und eine $q(n + t(n))$ -zeitbeschränkte TM M , die R simuliert.

Satz

Jede $t(n)$ -zeitbeschränkte TM kann durch eine RAM simuliert werden, die im logarithmischen Kostenmass durch $O((t(n) + n) \cdot \log(t(n) + n))$ zeitbeschränkt ist.

Also: TM und RAM im logarithmischen Kostenmodell können einander gegenseitig mit polynomielltem Zeitverlust simulieren

Vorlesung VL-12

P und NP

- Polynomielle Algorithmen
- Die Komplexitätsklasse P
- Nicht-deterministische Turingmaschinen
- Die Komplexitätsklasse NP
- Katalog von Problemen in NP
- P versus NP

Die **Komplexitätstheorie** versucht,
(entscheidbare!) algorithmische Probleme
nach ihrem Bedarf an Berechnungsressourcen zu klassifizieren
und sie in **Komplexitätsklassen** einzuteilen.

Berechnungsressourcen:

Rechenzeit und Speicherplatz (als Funktion der Eingabelänge)

Polynomielle Algorithmen

Definition

Die **Worst Case Laufzeit** $t_A(n)$ eines Algorithmus A misst die maximalen Laufzeitkosten auf Eingaben der Länge n bezüglich des logarithmischen Kostenmaßes der RAM.

Definition

Die Worst Case Laufzeit $t_A(n)$ eines Algorithmus A ist **polynomiell** beschränkt, falls gilt

$$\exists \alpha \in \mathbb{N} : t_A(n) \in O(n^\alpha)$$

Einen Algorithmus mit polynomiell beschränkter Worst Case Laufzeit bezeichnen wir als **polynomiellen Algorithmus** oder auch als **Polynomialzeitalgorithmus**.

Polynomiell beschränkt: $O(n)$; $O(n \log n)$; $O(n^3)$; $O(n^{100})$

Nicht polynomiell beschränkt: $O(2^n)$; $O(3^n)$; $O(n!)$; $O(2^{2^n})$

Beispiel: Sortieren

Problem: SORTIEREN

Eingabe: Zahlen $a_1, \dots, a_n \in \mathbb{N}$ in Binärdarstellung

Ausgabe: Die aufsteigend sortierte Folge der Eingabezahlen

Beispiel: Sortieren

Problem: SORTIEREN

Eingabe: Zahlen $a_1, \dots, a_n \in \mathbb{N}$ in Binärdarstellung

Ausgabe: Die aufsteigend sortierte Folge der Eingabezahlen

Satz

SORTIEREN kann in polynomieller Zeit gelöst werden.

Beispiel: Sortieren

Problem: SORTIEREN

Eingabe: Zahlen $a_1, \dots, a_n \in \mathbb{N}$ in Binärdarstellung

Ausgabe: Die aufsteigend sortierte Folge der Eingabezahlen

Satz

SORTIEREN kann in polynomieller Zeit gelöst werden.

Beweis:

- Wir lösen das Problem mit MergeSort oder HeapSort
- Laufzeit im uniformen Kostenmaß: $O(n \log n)$

Problem: SORTIEREN

Eingabe: Zahlen $a_1, \dots, a_n \in \mathbb{N}$ in Binärdarstellung

Ausgabe: Die aufsteigend sortierte Folge der Eingabezahlen

Satz

SORTIEREN kann in polynomieller Zeit gelöst werden.

Beweis:

- Wir lösen das Problem mit MergeSort oder HeapSort
- Laufzeit im uniformen Kostenmaß: $O(n \log n)$
- Laufzeit im logarithmischen Kostenmaß:
 - $O(\ell n \log n)$, wobei $\ell = \max_{1 \leq i \leq n} \log a_i$
 - Für die Gesamtlänge L der Eingabe gilt $L \geq \ell$ und $L \geq n$
 - Somit ist die Laufzeit durch $\ell n \log n \leq L^3$ beschränkt

Die folgenden Probleme können in polynomieller Zeit gelöst werden:

- Eulerkreis
- Kürzester Weg
- Minimaler Spannbaum
- Maximaler Fluss
- Maximum Matching
- Grösster gemeinsamer Teiler
- Konvexe Hülle in 2D

- Primzahltest

Definition: Komplexitätsklasse P

P ist die Klasse aller Entscheidungsprobleme,
für die es einen polynomiellen Algorithmus gibt.

Definition: Komplexitätsklasse P

P ist die Klasse aller Entscheidungsprobleme, für die es einen polynomiellen Algorithmus gibt.

Anmerkungen:

- **P** steht für **P**olynomiell
- **P** enthält ausschliesslich **Entscheidungsprobleme**
- Statt der RAM könnte man in der Definition der polynomiellen Laufzeit und der polynomiellen Algorithmen genauso gut die TM verwenden: RAM (im logarithmischem Kostenmaß) und TM simulieren einander ja mit polynomielltem Zeitverlust
- Polynomielle Algorithmen werden oft als **effiziente Algorithmen** bezeichnet, und **P** als die Klasse der **effizient lösbaren** Probleme.

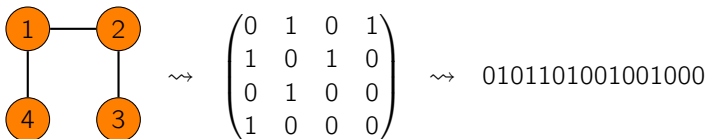
Beispiel: Graphzusammenhang (1)

Problem: Graphzusammenhang

Eingabe: Ein ungerichteter Graph $G = (V, E)$

Frage: Ist G zusammenhängend?

Anmerkung: Bei Graphproblemen gehen wir grundsätzlich davon aus, dass der Graph in Form einer Adjazenzmatrix gegeben ist.



Beispiel: Graphzusammenhang (2)

Satz

Graphzusammenhang $\in P$.

Beweis

- Wir lösen das Problem mit Tiefensuche (DFS).
- Laufzeit im uniformen Kostenmaß: $O(|V| + |E|) \subseteq O(|V|^2)$
Laufzeit im logarithmischen Kostenmaß: $O(|V|^2 \cdot \log |V|)$

Beispiel: Graphzusammenhang (2)

Satz

Graphzusammenhang $\in P$.

Beweis

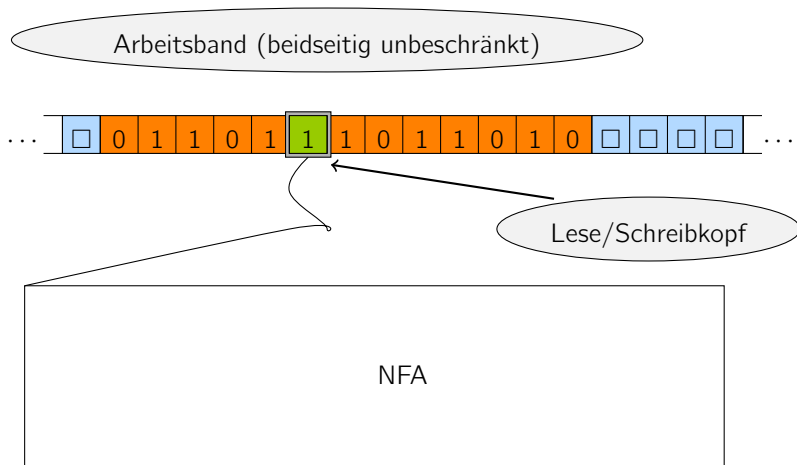
- Wir lösen das Problem mit Tiefensuche (DFS).
- Laufzeit im uniformen Kostenmaß: $O(|V| + |E|) \subseteq O(|V|^2)$
Laufzeit im logarithmischen Kostenmaß: $O(|V|^2 \cdot \log |V|)$
- Die Gesamtlänge der Eingabe ist $L \in \Theta(|V|^2)$.
Die Gesamtlaufzeit liegt somit in

$$O(|V|^2 \log |V|) \subseteq O(L \log L) \subseteq O(L^2)$$

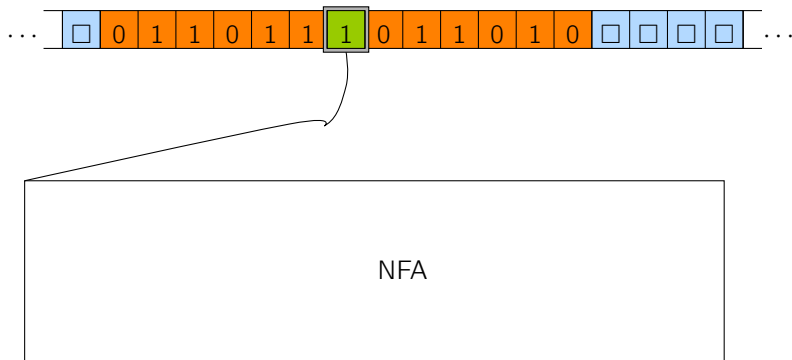
Die nicht-deterministische Turingmaschine (NTM)

Turing Award 1976 to Michael O. Rabin and Dana S. Scott,
*for their joint paper “Finite Automata and Their Decision Problem”,
which introduced the idea of **non-deterministic machines**, which has
proved to be an enormously valuable concept.*

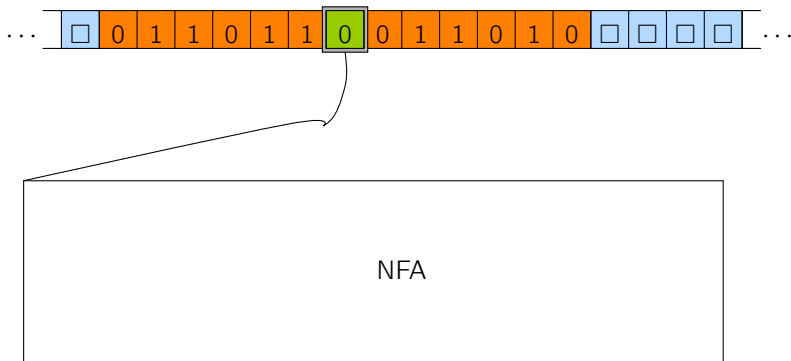
Illustration



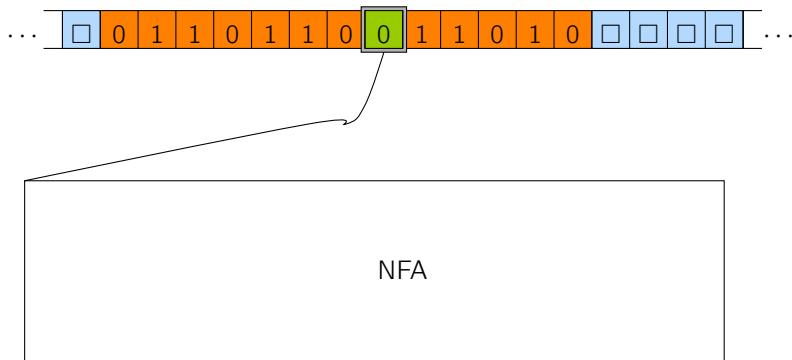
Illustration



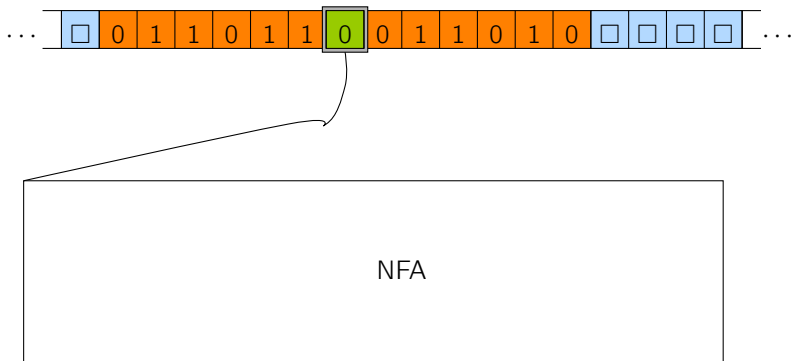
Illustration



Illustration



Illustration



Definition: Nicht-deterministische Turingmaschine (NTM)

Eine nicht-deterministische Turingmaschine (NTM) verfügt über

- ein beidseitig unbeschränktes Arbeitsband,
- einen Lese/Schreibkopf, und
- einen Mechanismus, der die Zustandsüberführungen steuert.

Definition: Nicht-deterministische Turingmaschine (NTM)

Eine nicht-deterministische Turingmaschine (NTM) verfügt über

- ein beidseitig unbeschränktes Arbeitsband,
- einen Lese/Schreibkopf, und
- einen Mechanismus, der die Zustandsüberführungen steuert.

Der einzige Unterschied zur deterministischen Turingmaschine TM besteht darin, dass die Zustandsüberführungen bei der NTM nicht durch eine **Funktion** sondern durch eine **Relation** gesteuert werden:

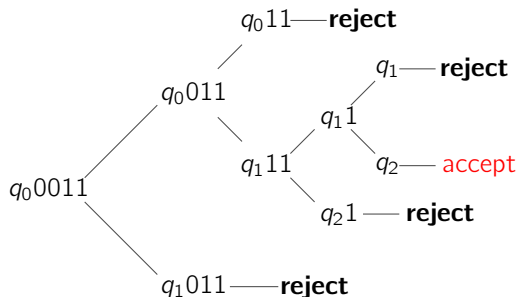
$$\delta \subseteq ((Q \setminus \{\bar{q}\}) \times \Gamma) \times (Q \times \Gamma \times \{L, R, N\})$$

Berechnungen auf einer NTM sind **nicht deterministisch**, da es zu einer Konfiguration mehrere direkte Nachfolgekongfigurationen geben kann.

Berechnungsbaum (1)

δ	0	1	B
q_0	$\{(q_0, B, R), (q_1, B, R)\}$	{reject}	{reject}
q_1	{reject}	$\{(q_1, B, R), (q_2, B, R)\}$	{reject}
q_2	{reject}	{reject}	{accept}

Diese NTM hat für Eingabe $w = 0011$ den folgenden Berechnungsbaum:



Berechnungsbaum (2)

Rechenweg einer NTM = Konfigurationsfolge, die mit Startkonfiguration beginnt und mit Nachfolgekongfigurationen fortgesetzt wird, bis Endkonfiguration im Zustand $\bar{q}$ erreicht wird

Berechnungsbaum (2)

Rechenweg einer NTM = Konfigurationsfolge, die mit Startkonfiguration beginnt und mit Nachfolgekonfigurationen fortgesetzt wird, bis Endkonfiguration im Zustand $\bar{q}$ erreicht wird

Die möglichen Rechenwege einer NTM M auf einer Eingabe können in einem **Berechnungsbaum** zusammengefasst werden:

- Die Knoten des Baumes entsprechen Konfigurationen
- Die Wurzel des Baumes entspricht der Startkonfiguration
- Die Kinder einer Konfiguration entsprechen den möglichen Nachfolgekonfigurationen

Der **maximale Verzweigungsgrad** des Berechnungsbaumes ist

$$\Delta := \max \{ |\delta(q, a)| \mid q \in Q \setminus \{\bar{q}\}, a \in \Gamma \}.$$

Anmerkung: Δ hängt nur von M und nicht von der Eingabe ab

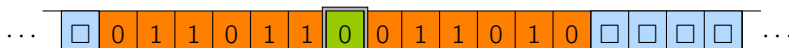
Akzeptanzverhalten (1)

Definition: Akzeptanzverhalten der NTM

Eine NTM M akzeptiert die Eingabe $x \in \Sigma^*$,
falls es **mindestens** einen Rechenweg von M gibt,
der in eine Konfiguration mit akzeptierendem Zustand führt.

Die von der NTM M erkannte Sprache $L(M)$
besteht aus allen von M akzeptierten Wörtern.

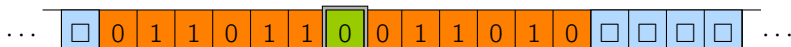
Akzeptanzverhalten (2)



δ	0	1	B
q_0	$\{(q_0, B, R), (q_1, B, R)\}$	{reject}	{reject}
q_1	{reject}	$\{(q_1, B, R), (q_2, B, R)\}$	{reject}
q_2	{reject}	{reject}	{accept}

Frage: Welche Sprache wird von dieser NTM erkannt?

Akzeptanzverhalten (2)



δ	0	1	B
q_0	$\{(q_0, B, R), (q_1, B, R)\}$	{reject}	{reject}
q_1	{reject}	$\{(q_1, B, R), (q_2, B, R)\}$	{reject}
q_2	{reject}	{reject}	{accept}

Frage: Welche Sprache wird von dieser NTM erkannt?

Antwort: $L(M) = \{0^i 1^j \mid i, j \geq 1\}$

Definition: Laufzeit der NTM

Die **Laufzeit** einer NTM M auf einer Eingabe x ist wie folgt definiert.

- Falls $x \in L(M)$, so ist die Laufzeit $T_M(x)$ die Länge des kürzesten akzeptierenden Rechenweges von M auf x .
- Falls $x \notin L(M)$, so ist die Laufzeit $T_M(x) = 0$.

Definition: Laufzeit der NTM

Die **Laufzeit** einer NTM M auf einer Eingabe x ist wie folgt definiert.

- Falls $x \in L(M)$, so ist die Laufzeit $T_M(x)$ die Länge des kürzesten akzeptierenden Rechenweges von M auf x .
- Falls $x \notin L(M)$, so ist die Laufzeit $T_M(x) = 0$.

Die **Worst Case Laufzeit** $t_M(n)$ der NTM M auf Eingaben der Länge $n \in \mathbb{N}$ ist definiert als $t_M(n) := \max\{T_M(x) \mid x \in \Sigma^n\}$

Die Komplexitätsklasse NP

Definition: Komplexitätsklasse NP

NP ist die Klasse aller Entscheidungsprobleme, die durch eine NTM M erkannt werden, deren Worst Case Laufzeit $t_M(n)$ polynomiell beschränkt ist.

NP steht für **N**icht-deterministisch **P**olynomiell

Problem: CLIQUE

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl k

Frage: Enthält G eine Clique mit $\geq k$ Knoten?

Clique = Menge $S \subseteq V$ von paarweise benachbarten Knoten

Problem: CLIQUE

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl k

Frage: Enthält G eine Clique mit $\geq k$ Knoten?

Clique = Menge $S \subseteq V$ von paarweise benachbarten Knoten

Satz

CLIQUE $\in$ NP

Problem: CLIQUE

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl k

Frage: Enthält G eine Clique mit $\geq k$ Knoten?

Clique = Menge $S \subseteq V$ von paarweise benachbarten Knoten

Satz

CLIQUE $\in$ NP

Beweis: Wir beschreiben eine NTM M mit $L(M) = \text{CLIQUE}$.

- Syntaktisch inkorrekte Eingaben werden verworfen
- M rät nicht-deterministisch einen 0-1-String y der Länge $|V|$
- M akzeptiert, falls der String y mindestens k Einsen enthält und falls die Knotenmenge $C = \{i \in V \mid y_i = 1\}$ eine Clique bildet

Korrektheit: $\exists$ akzeptierender Rechenweg $\iff G$ enthält k -Clique

Laufzeit: Jede Phase kostet polynomielle Zeit

Alternative Charakterisierung von NP

Satz (Zertifikat Charakterisierung von NP)

Eine Sprache $L \subseteq \Sigma^*$ liegt genau dann in NP,
wenn es einen polynomiellen **deterministischen** Algorithmus V
und ein Polynom p mit der folgenden Eigenschaft gibt:

$$x \in L \iff \exists y \in \{0, 1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$$

Anmerkungen:

- Der polynomielle Algorithmus V wird auch **Verifizierer** genannt
- Das Wort $y \in \{0, 1\}^*$ wird auch **Zertifikat** genannt

Beweis: NTM $\Rightarrow$ Zertifikat

$$x \in L \iff \exists y \in \{0,1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$$

- Es sei M eine NTM, die L in polynomieller Zeit erkennt
- Die Laufzeit von M sei beschränkt durch ein Polynom q
- Der maximale Verzweigungsgrad eines Berechnungsbaumes sei Δ

Beweis: NTM $\Rightarrow$ Zertifikat

$$x \in L \iff \exists y \in \{0,1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$$

- Es sei M eine NTM, die L in polynomieller Zeit erkennt
- Die Laufzeit von M sei beschränkt durch ein Polynom q
- Der maximale Verzweigungsgrad eines Berechnungsbaumes sei Δ

Konstruktion von Zertifikat und Verifizierer:

- Für die Eingabe $x \in L$ betrachten wir den kürzesten akzeptierenden Rechenweg im Berechnungsbaum

Beweis: NTM $\Rightarrow$ Zertifikat

$$x \in L \iff \exists y \in \{0,1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$$

- Es sei M eine NTM, die L in polynomieller Zeit erkennt
- Die Laufzeit von M sei beschränkt durch ein Polynom q
- Der maximale Verzweigungsgrad eines Berechnungsbaumes sei Δ

Konstruktion von Zertifikat und Verifizierer:

- Für die Eingabe $x \in L$ betrachten wir den kürzesten akzeptierenden Rechenweg im Berechnungsbaum
- Das Zertifikat y kodiert den akzeptierenden Rechenweg Schritt für Schritt, mit $\leq \log_2 \Delta$ Bits pro Verzweigung

Beweis: NTM $\Rightarrow$ Zertifikat

$$x \in L \iff \exists y \in \{0,1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$$

- Es sei M eine NTM, die L in polynomieller Zeit erkennt
- Die Laufzeit von M sei beschränkt durch ein Polynom q
- Der maximale Verzweigungsgrad eines Berechnungsbaumes sei Δ

Konstruktion von Zertifikat und Verifizierer:

- Für die Eingabe $x \in L$ betrachten wir den kürzesten akzeptierenden Rechenweg im Berechnungsbaum
- Das Zertifikat y kodiert den akzeptierenden Rechenweg Schritt für Schritt, mit $\leq \log_2 \Delta$ Bits pro Verzweigung
- Das Zertifikat hat polynomielle Länge $|y| \leq \log_2 \Delta \cdot q(|x|)$
- Der Verifizierer V erhält $y\#x$ und simuliert den Rechenweg der NTM M für die Eingabe x

Beweis: Zertifikat $\Rightarrow$ NTM

$$x \in L \iff \exists y \in \{0, 1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$$

- Es sei V ein Verifizierer mit polynomieller Laufzeitschranke
- Das Polynom p beschränkt die Länge des Zertifikats

Beweis: Zertifikat $\Rightarrow$ NTM

$$x \in L \iff \exists y \in \{0, 1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$$

- Es sei V ein Verifizierer mit polynomieller Laufzeitschranke
- Das Polynom p beschränkt die Länge des Zertifikats

Konstruktion von NTM:

- Für eine Eingabe x rät M zunächst nicht-deterministisch das Zertifikat $y \in \{0, 1\}^*$ mit $|y| \leq p(|x|)$
- Das Zertifikat wird in polynomieller Zeit $p(|x|)$ geraten.

Beweis: Zertifikat $\Rightarrow$ NTM

$$x \in L \iff \exists y \in \{0, 1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$$

- Es sei V ein Verifizierer mit polynomieller Laufzeitschranke
- Das Polynom p beschränkt die Länge des Zertifikats

Konstruktion von NTM:

- Für eine Eingabe x rät M zunächst nicht-deterministisch das Zertifikat $y \in \{0, 1\}^*$ mit $|y| \leq p(|x|)$
- Das Zertifikat wird in polynomieller Zeit $p(|x|)$ geraten.
- Dann simuliert M den Verifizierer V auf dem Wort $y\#x$, und akzeptiert wenn der Verifizierer akzeptiert
- Die Zeit für die Simulation ist polynomiell beschränkt in der polynomiellen Laufzeit des Verifizierers.

Katalog von Problemen in NP

Probleme in NP (1): Satisfiability

Problem: Satisfiability (SAT)

Eingabe: Eine Boole'sche Formel φ in CNF über einer Boole'schen Variablenmenge $X = \{x_1, \dots, x_n\}$

Frage: Existiert eine Wahrheitsbelegung von X , die φ erfüllt?

Literal = positive oder negierte Variable

Klausel = ODER-Verknüpfung von einigen Literalen

Beispiele:

- $\varphi_1 = (x \vee y \vee z) \wedge (\neg x \vee \neg y \vee \neg z)$
 $\varphi_1 = (x + y + z)(\bar{x} + \bar{y} + \bar{z})$
- $\varphi_2 = (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$
 $\varphi_2 = (x + y)(\bar{x} + y)(x + \bar{y})(\bar{x} + \bar{y})$

Probleme in NP (1): Satisfiability

Problem: Satisfiability (SAT)

Eingabe: Eine Boole'sche Formel φ in CNF über einer Boole'schen Variablenmenge $X = \{x_1, \dots, x_n\}$

Frage: Existiert eine Wahrheitsbelegung von X , die φ erfüllt?

Literal = positive oder negierte Variable

Klausel = ODER-Verknüpfung von einigen Literalen

Beispiele:

- $\varphi_1 = (x \vee y \vee z) \wedge (\neg x \vee \neg y \vee \neg z)$
 $\varphi_1 = (x + y + z)(\bar{x} + \bar{y} + \bar{z})$
- $\varphi_2 = (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$
 $\varphi_2 = (x + y)(\bar{x} + y)(x + \bar{y})(\bar{x} + \bar{y})$

Frage: Wie sieht ein NP-Zertifikat für SAT aus?

Probleme in NP (2): Clique / Independent Set / VC

Problem: CLIQUE

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl k

Frage: Enthält G eine Clique mit $\geq k$ Knoten?

Problem: Independent Set (INDEP-SET)

Frage: Enthält G eine unabhängige Menge mit $\geq k$ Knoten?

Problem: Vertex Cover (VC)

Frage: Enthält G ein Vertex Cover mit $\leq k$ Knoten?

- Unabhängige Menge (independent set) $S \subseteq V$ spannt keine Kanten
- Vertex Cover $S \subseteq V$ berührt alle Kanten

Probleme in NP (2): Clique / Independent Set / VC

Problem: CLIQUE

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl k

Frage: Enthält G eine Clique mit $\geq k$ Knoten?

Problem: Independent Set (INDEP-SET)

Frage: Enthält G eine unabhängige Menge mit $\geq k$ Knoten?

Problem: Vertex Cover (VC)

Frage: Enthält G ein Vertex Cover mit $\leq k$ Knoten?

- Unabhängige Menge (independent set) $S \subseteq V$ spannt keine Kanten
- Vertex Cover $S \subseteq V$ berührt alle Kanten

Frage: Wie sieht NP-Zertifikat für CLIQUE / INDEP-SET / VC aus?

Probleme in NP (3): Hamiltonkreis / TSP

Problem: Hamiltonkreis (Ham-Cycle)

Eingabe: Ein ungerichteter Graph $G = (V, E)$

Frage: Besitzt G einen Hamiltonkreis?

Problem: Travelling Salesman Problem (TSP)

Eingabe: Städte $1, \dots, n$; Distanzen $d(i, j)$; eine Zahl γ

Frage: Gibt es eine Rundreise (TSP-Tour) mit Länge höchstens γ ?

Probleme in NP (3): Hamiltonkreis / TSP

Problem: Hamiltonkreis (Ham-Cycle)

Eingabe: Ein ungerichteter Graph $G = (V, E)$

Frage: Besitzt G einen Hamiltonkreis?

Problem: Travelling Salesman Problem (TSP)

Eingabe: Städte $1, \dots, n$; Distanzen $d(i, j)$; eine Zahl γ

Frage: Gibt es eine Rundreise (TSP-Tour) mit Länge höchstens γ ?

Fragen:

Wie sieht ein NP-Zertifikat für Ham-Cycle aus?

Wie sieht ein NP-Zertifikat für TSP aus?

Probleme in NP (4): Exact Cover

Problem: Exact Cover (EX-COVER)

Eingabe: Eine endliche Menge X ; Teilmengen $S_1, \dots, S_m$ von X

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, m\}$,
sodass die Mengen S_i mit $i \in I$ eine Partition von X bilden?

Frage: Wie sieht ein NP-Zertifikat für Exact Cover aus?

Probleme in NP (5): SUBSET-SUM / PARTITION

Problem: SUBSET-SUM

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$; eine ganze Zahl b

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a_i = b$?

Problem: PARTITION

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$; mit $\sum_{i=1}^n a_i = 2A$

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a_i = A$?

Probleme in NP (5): SUBSET-SUM / PARTITION

Problem: SUBSET-SUM

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$; eine ganze Zahl b

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a_i = b$?

Problem: PARTITION

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$; mit $\sum_{i=1}^n a_i = 2A$

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a_i = A$?

Fragen:

Wie sieht ein NP-Zertifikat für SUBSET-SUM aus?

Wie sieht ein NP-Zertifikat für PARTITION aus?

P versus NP

Übung

Beweisen Sie, dass $P \subseteq NP$ gilt

Argument 1: Über TM und NTM

Argument 2: Wie sieht das NP-Zertifikat für ein Problem in P aus?

Was haben wir bis jetzt gesehen?

Die Komplexitätsklasse P

Die Klasse P enthält alle Entscheidungsprobleme,
die effizient auf dem Computer gelöst werden können

Intuitiv: P enthält die Probleme, die wir gut verstehen können und die wir in vernünftiger Rechenzeit erledigen können

Die Komplexitätsklasse NP

Die Klasse NP enthält alle Entscheidungsprobleme,
für die eine kurze Lösung **existiert**,
und deren kurze Lösung wir effizient überprüfen können
(wenn wir diese Lösung erst einmal gezeigt bekommen)

Intuitiv: NP enthält alle natürlichen Probleme, die nach einem konkreten Lösungsobjekt fragen

P=NP ?

Falls die Lösung eines Problems einfach zu **überprüfen** ist,
ist es dann auch immer einfach, die Lösung selbst zu **entdecken**?

Falls $P=NP$ gilt:

- Viele schwierige Probleme aus Wirtschaft und Industrie können schnell gelöst werden
- Perfekte Fahrpläne, perfekte Produktionspläne, perfekte Transportpläne, etc
- Die Mathematik erreicht eine völlig neue Stufe: Wenn es für ein Theorem einen kurzen Beweis gibt, so können wir diesen Beweis auch schnell finden
- Die moderne Kryptographie bricht zusammen

Falls $P \neq NP$ gilt:

- Schwierige Probleme aus Wirtschaft und Industrie können nur durch viel Rechenzeit und Expertenwissen attackiert werden
- Perfekte Lösungen für schwierige Optimierungsprobleme mit grossen Datenmengen sind nicht zu erwarten
- Die Mathematik arbeitet genauso weiter wie bisher
- Die Kryptographie arbeitet genauso weiter wie bisher

(Default scenario, in dem die Informatik momentan arbeitet)

Eine Million Dollar Preisgeld

Im Jahr 2000 hat das **Clay Mathematics Institute (CMI)** je eine Million Dollar Preisgeld für die Lösung der folgenden sieben Probleme ausgelobt:

- P versus NP Problem
- Hodge Vermutung
- Poincaré Vermutung (erledigt 2002–2006; Grigori Perelman)
- Riemann'sche Vermutung
- Quantenversion der Yang-Mills Gleichungen
- Navier-Stokes Gleichungen
- Birch und Swinnerton-Dyer Vermutung

https://en.wikipedia.org/wiki/Millennium_Prize_Problems